

## OpenPulse Project Information

Generated: 2026-05-15T06:36:53.597+00:00

### PROJECT OVERVIEW

Name: OpenPulse

Description: AI-powered autonomous news-to-video generation pipeline using HyperFrames and remote LLM inference.

Major Capabilities: AI Video Generation, Autonomous Agents, News Ingestion

### TECHNOLOGY STACK (from package.json)

Runtime: Node.js (module modules)

Core Scripts:

- dev: tsx server.ts
- build: vite build
- start: tsx server.ts
- clean: rm -rf dist
- lint: tsc --noEmit

### PRODUCTION DEPENDENCIES

- @fastify/cors: ^10.1.0
- @fastify/static: ^8.3.0
- @fastify/view: ^10.0.2
- @tailwindcss/vite: ^4.2.4
- @types/react: ^19.2.14
- @types/react-dom: ^19.2.3
- @vitejs/plugin-react: ^5.2.0
- axios: ^1.16.0
- dotenv: ^17.2.3
- express: ^5.2.1
- fastify: ^5.8.5
- form-data: ^4.0.1
- fs-extra: ^11.3.5
- gsap: ^3.15.0
- hyperframes: ^0.5.4
- lucide-react: ^0.546.0
- motion: ^12.38.0
- pino: ^9.14.0
- pino-pretty: ^11.3.0
- playwright: ^1.59.1
- react: ^19.2.6
- react-dom: ^19.2.6
- rss-parser: ^3.13.0

### DEVELOPMENT DEPENDENCIES

- @types/express: ^4.17.25
- @types/node: ^22.14.0
- autoprefixer: ^10.4.21
- tailwindcss: ^4.1.14
- tsx: ^4.21.0
- typescript: ~5.8.2
- vite: ^6.2.3

### TOP-LEVEL REPOSITORY STRUCTURE

- .env.example (file)
- .gitignore (file)
- README.md (file)
- app (dir)
- assets (dir)
- index.html (file)
- metadata.json (file)
- package-lock.json (file)
- package.json (file)
- server.ts (file)
- src (dir)
- templates (dir)
- tsconfig.json (file)
- vite.config.ts (file)

## README CONTENT

# OpenPulse 1.0.0

AI-powered autonomous news-to-video generation pipeline. This system transforms real-time news feeds into cinematic vertical short-form videos using an orchestrated pipeline of LLMs, GSAP animations, and HyperFrames rendering.

### ## ð Architecture

1. **Ingestion**: Scrapes Hacker News (and other sources) for trending stories.
2. **Inference**: Uses vLLM or LM Studio (Local) to generate a HyperFrames-compatible HTML composition.
3. **Composition**: Dynamically builds a GSAP-powered animation sequence.
4. **Rendering**: Uses Headless Chromium via Playwright to capture frames.
5. **Encoding**: FFMPEG compiles frames into a production-ready MP4.

### ## ð ð Tech Stack

- **Backend**: Node.js, Fastify, TypeScript
- **Frontend**: React, TailwindCSS, Lucide-React, Motion
- **Inference**: **vLLM** (Production-grade), LM Studio (Local / Mesh)
- **Rendering**: [HeyGen HyperFrames](https://github.com/heygen-com/hyperframes)
- **Animation**: GSAP (GreenSock)

### ## ð f Setup & Installation

#### ### 1. vLLM (Recommended for Production)

1. Install vLLM following the [official guide](https://github.com/vllm-project/vllm).
2. Run the server:
 

```
““bash
python -m vllm.entrypoints.openai.api_server --model Qwen/Qwen2.5-7B-Instruct --port 8000
““
```

#### ### 2. Application Setup

```
““bash
# Install dependencies
npm install
```

```
# Setup environment
cp .env.example .env
```

```
# Start development server
npm run dev
""
```

## ## ðŸ”¥ HyperFrames Integration

OpenPulse treats HTML/JS as executable video code.

- **\*\*Deterministic\*\***: Every frame is calculated based on the GSAP timeline.
- **\*\*Clip-Safe\*\***: Uses 'data-start' and 'data-duration' attributes for precise temporal alignment.

## ## ðŸ”Œ Security

- Encrypted mesh networking via Tailscale (integrated via LM Link).
- Sandboxed rendering environment.
- Environment-based secret management.